VISIONINSPECT AI

Manufacturing Defect Detection & Automated Quality Inspection Platform

FINAL PROJECT DOCUMENTATION

Milestones 1–4

Infosys Springboard Internship Project

Project Information	Details
Project	VisionInspect AI
Duration	Milestones 1–4 / Weeks 1–8
Backend	Python, FastAPI, Uvicorn
Frontend	React.js, Vite, Nginx
Database	PostgreSQL
AI/CV	PyTorch, Torchvision, ResNet18 feature extraction, OpenCV, NumPy, Scikit-learn
Dataset	MVTec Anomaly Detection – 15 categories
Deployment	Docker Compose on Microsoft Azure VM
Version Control	Git and GitHub
Final Status	End-to-end platform deployed and demonstrated

2026

Table of Contents

1. Executive Summary

2. Project Background and Objectives

3. Requirements and Scope

4. Overall System Architecture

5. Technology Stack

6. Milestone 1 – Initialization, Design & Core Setup

7. Milestone 2 – Image Processing & Defect Detection

8. Milestone 3 – Defect Classification & Manufacturing Analytics

9. Milestone 4 – Testing, Deployment & Documentation

10. AI/CV Methodology

11. Dataset and Evaluation Protocol

12. Model Validation Results

13. Functional and System Testing

14. User Interface and Functional Evidence

15. Docker and Azure Deployment

16. Reports and Analytics

17. Limitations and Future Enhancements

18. Final End-to-End Demonstration

19. Milestone Completion Matrix

20. Conclusion

1. Executive Summary

VisionInspect AI is an AI-powered manufacturing quality inspection platform designed to automate visual inspection of manufactured products. The platform accepts product images, performs computer-vision based anomaly detection and defect classification, assigns severity and a quality-control decision, stores inspection records, and provides dashboards, analytics and downloadable reports.

The project was implemented progressively across four milestones. Milestone 1 established the project architecture, authentication, dataset integration and inspection interface. Milestone 2 implemented image processing and anomaly/defect detection. Milestone 3 added defect classification, severity assessment, quality control, reporting and manufacturing analytics. Milestone 4 focused on validation, optimization, Docker/cloud deployment, documentation and final demonstration.

2. Project Background and Objectives

The project specification identifies the following major objectives:

- Implement image processing and defect detection systems.
- Build AI-powered computer-vision inspection workflows.
- Understand image processing and anomaly detection concepts.
- Generate real-time defect detection insights.
- Implement defect categorization and severity assessment.
- Generate production quality reports and manufacturing analytics.
- Build trend monitoring and quality-control workflows.
- Deploy the platform using Docker and a cloud environment.
- Prepare technical documentation and demonstrate the complete platform.

These objectives are aligned with the four-week-pair milestones defined in the project requirements. 毅filecite孳turn21file0孳L16-L45傢

3. Requirements and Scope

Milestone	Week	Primary Scope	Expected Outcome
Milestone 1	1–2	Initialization, design and core setup	Working authentication and inspection management system
Milestone 2	3–4	Image processing and defect detection	AI-powered inspection and real-time defect insights
Milestone 3	5–6	Defect classification and manufacturing analytics	Classification, quality control, reporting and analytics
Milestone 4	7–8	Testing, deployment and documentation	Validated, deployed and demonstrated platform

The official milestone structure specifies Milestone 1 as project initialization/core setup, Milestone 2 as image processing/defect detection, Milestone 3 as classification/analytics, and Milestone 4 as testing/deployment/documentation. 毅filecite孳turn21file1孳L202-L265傢

4. Overall System Architecture

The final architecture consists of a React frontend served by Nginx, a FastAPI backend responsible for authentication and AI inspection APIs, PostgreSQL for persistent data, and the MVTec dataset mounted into the backend container. Docker Compose orchestrates the services on an Azure virtual machine.

Layer	Technology	Responsibilities
Frontend	React.js / Vite / Nginx	Authentication UI, inspection workflow, dashboards, history, analytics and reports
API	FastAPI / Uvicorn	Authentication, inspection APIs, analytics, reporting and database access

AI/CV	PyTorch / Torchvision / ResNet18 / OpenCV	Feature extraction, anomaly scoring and defect classification
Database	PostgreSQL	Users, roles and inspection records
Dataset	MVTec AD	Normal and defective product images across 15 categories
Deployment	Docker Compose / Azure VM	Container orchestration and public hosting

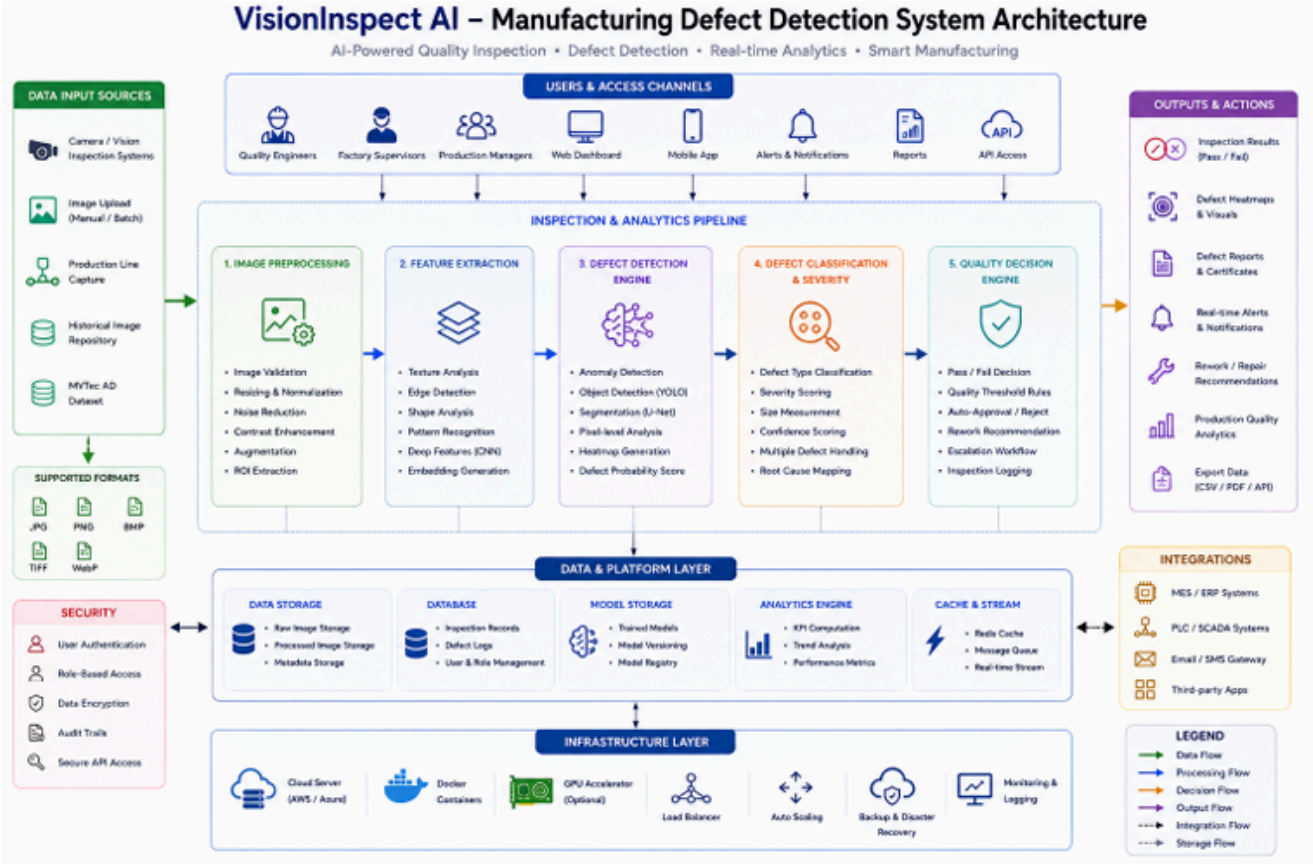


Figure 1. Overall system architecture diagram

5. Technology Stack

Area	Technologies
Programming	Python, JavaScript
Backend	FastAPI, Uvicorn
Frontend	React.js, Vite, Nginx
Database	PostgreSQL
AI/CV	PyTorch, Torchvision, ResNet18, OpenCV, NumPy, Scikit-learn
Authentication	JWT-based authentication and role-based access
Deployment	Docker, Docker Compose, Microsoft Azure VM
Version Control	Git, GitHub
Reporting	PDF and CSV generation

The project requirements list FastAPI, React.js, PostgreSQL, OpenCV, PyTorch, Scikit-learn, Docker, Azure/AWS and Git/GitHub among the expected technology options.

6. Milestone 1 – Initialization, Design & Core Setup

Milestone 1 covered Weeks 1 and 2 and established the foundation of VisionInspect AI.

6.1 Work Completed

- Defined the manufacturing quality-inspection workflow and project objectives.
- Designed the application architecture and database structure.
- Set up the frontend and backend development environments.
- Implemented authentication and role-based access.
- Integrated the MVTec Anomaly Detection dataset.
- Implemented image upload and inspection-management workflows.
- Created the initial inspection dashboard.

6.2 Milestone 1 Output

Requirement	Implementation Status
Project initialization	Completed
Architecture and database design	Completed
Authentication and role management	Completed
MVTec dataset integration	Completed
Image upload workflow	Completed
Initial inspection dashboard	Completed

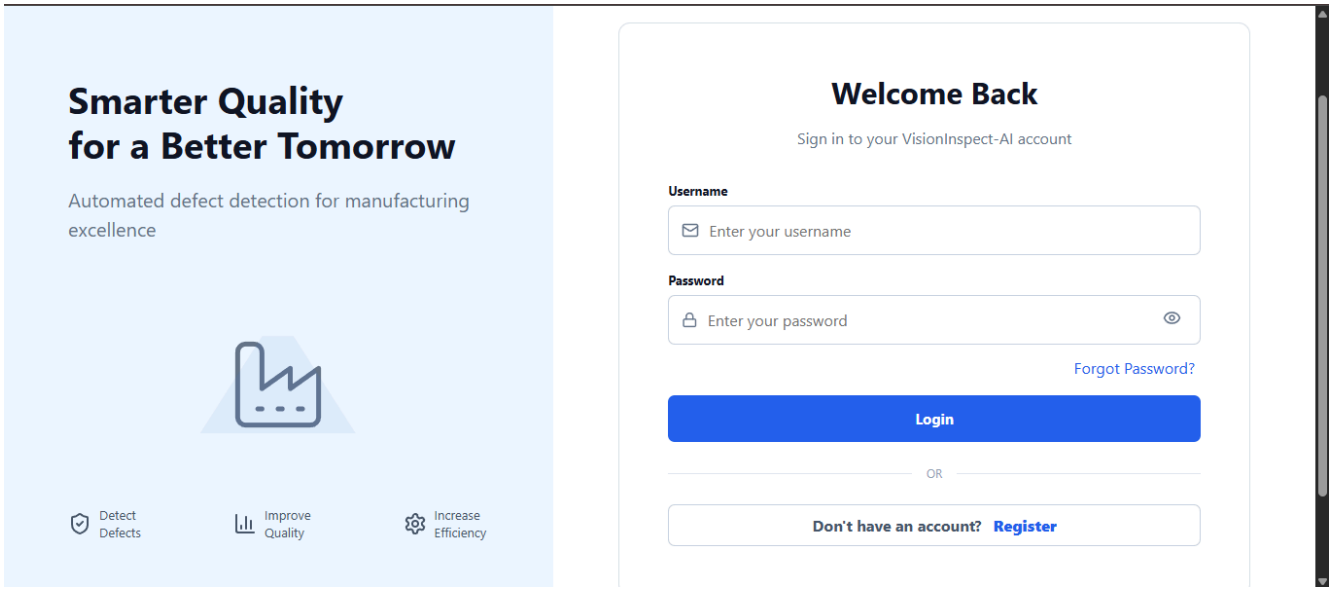


Figure 2. Login and authentication interface

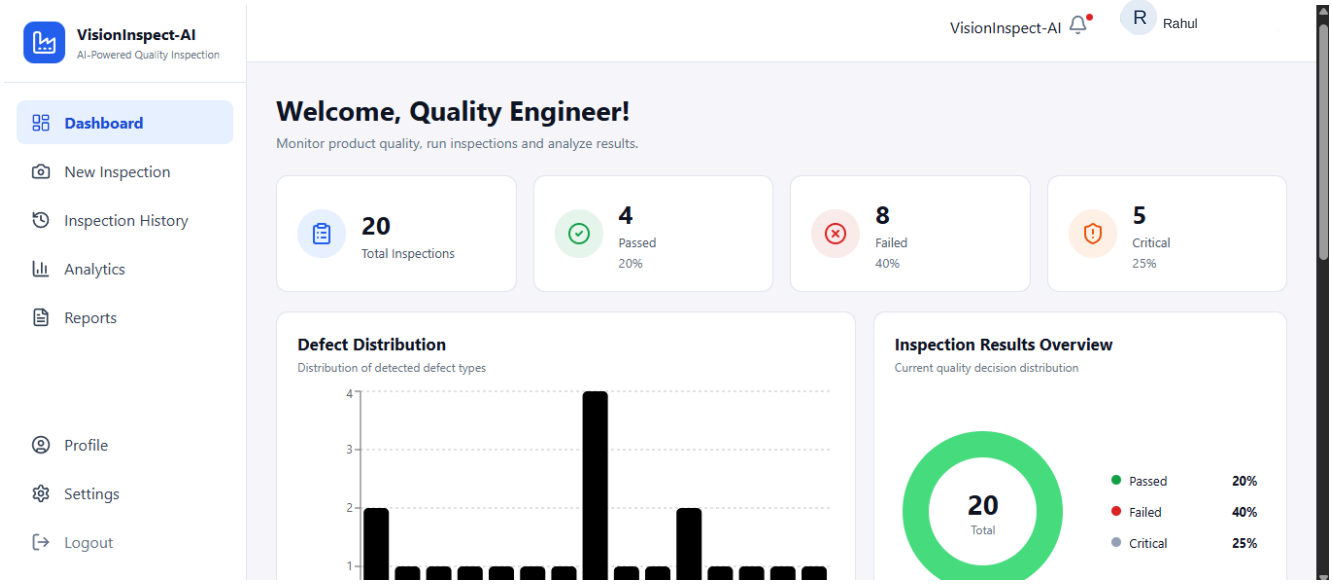


Figure 3. Initial dashboard / inspection interface

7. Milestone 2 – Image Processing & Defect Detection

Milestone 2 covered Weeks 3 and 4 and introduced the computer-vision inspection pipeline.

7.1 Work Completed

- Implemented image preprocessing and validation.
- Implemented image-quality analysis functionality.
- Built image analytics workflows.
- Integrated the defect/anomaly detection engine.
- Implemented defect prediction from product images.
- Connected AI inspection results to the inspection dashboard.

The project specification requires image preprocessing, image-quality reports, image analytics, defect detection, predictions and inspection monitoring in Milestone 2. 毅filecite孿turn21file1孿L222-L234傢

7.2 AI Detection Implementation

The implemented detector uses pretrained ResNet18 as a feature extractor. Images are represented in feature space and compared with reference images from the selected MVTec category. A category-specific threshold is used to determine whether the query is anomalous.

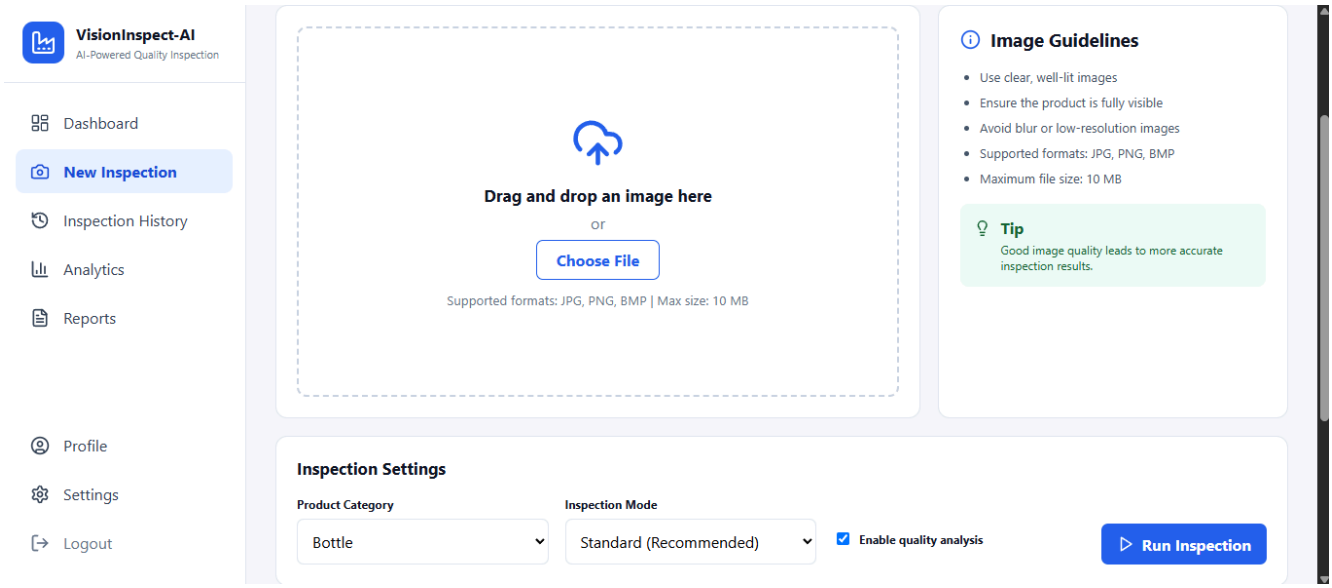


Figure 4. Image upload and inspection configuration

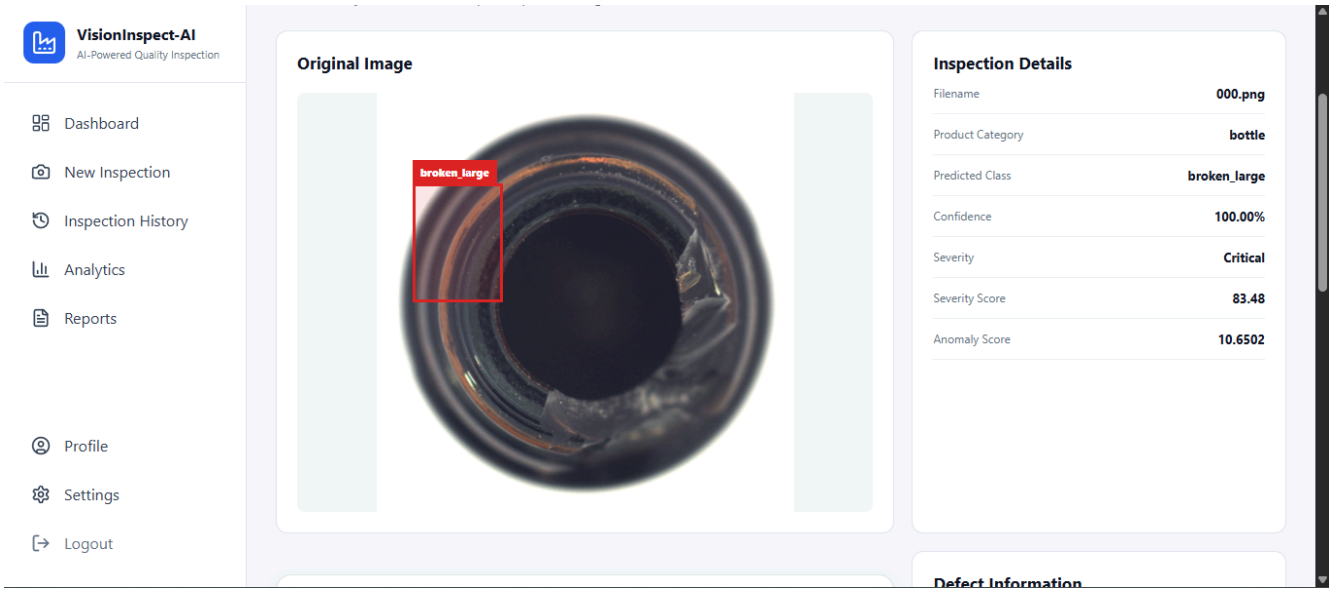


Figure 5. Defect/anomaly inspection output

8. Milestone 3 – Defect Classification & Manufacturing Analytics

Milestone 3 covered Weeks 5 and 6 and expanded the detector into a complete quality-assessment and analytics workflow.

8.1 Work Completed

- Implemented defect categorization/classification workflows.
- Implemented severity scoring and quality-risk assessment.
- Implemented quality-control decision logic.
- Generated production-quality reports.
- Built defect analytics dashboards.
- Implemented trend monitoring.
- Implemented inspection history and stored inspection records.

- Added role-specific Quality Engineer and Factory Supervisor views.

The project specification explicitly requires defect categorization, severity/risk assessment, quality modules, production reports, analytics dashboards and trend monitoring in Milestone 3.

8.2 Severity Scoring Framework

Factor	Weight
Defect Size	30%
Defect Location	25%
Defect Type	25%
Detection Confidence	20%

Severity levels are defined as Critical (80–100), High (60–79), Medium (40–59), and Low (0–39).

8.3 Quality Control and Analytics

- Pass/fail/review quality decisions.
- Severity-based quality assessment.
- Inspection history with searchable records.
- Defect distribution and quality trend analytics.
- Production-quality PDF report.
- Inspection data CSV export.
- Factory Supervisor production dashboard.

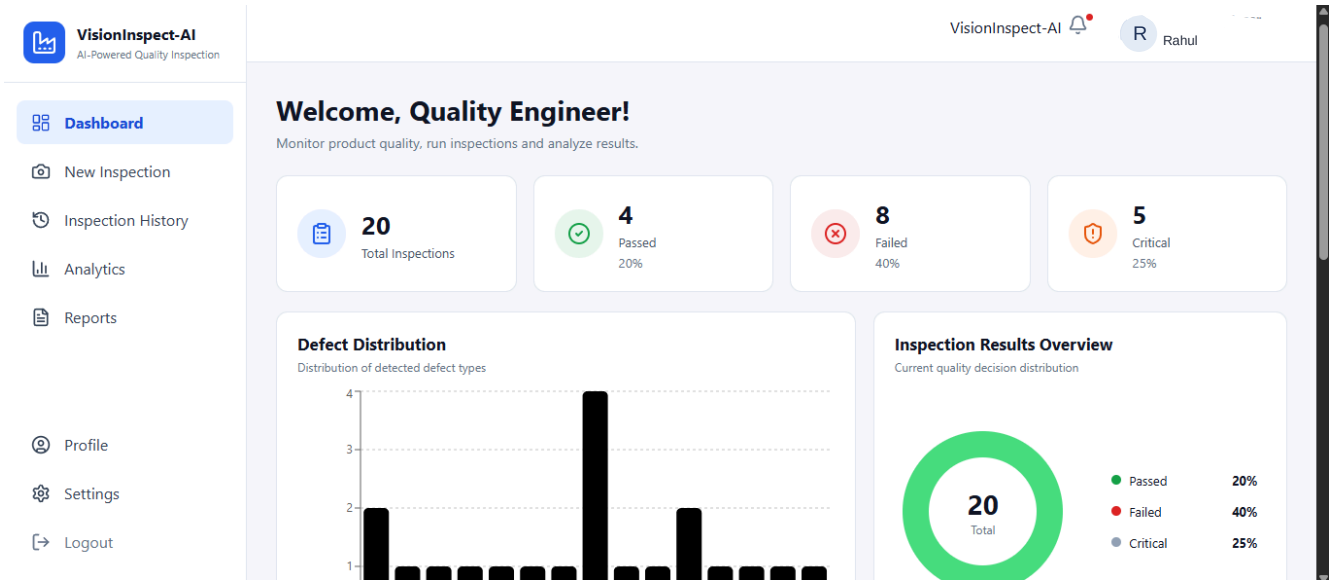


Figure 6. Quality Engineer dashboard

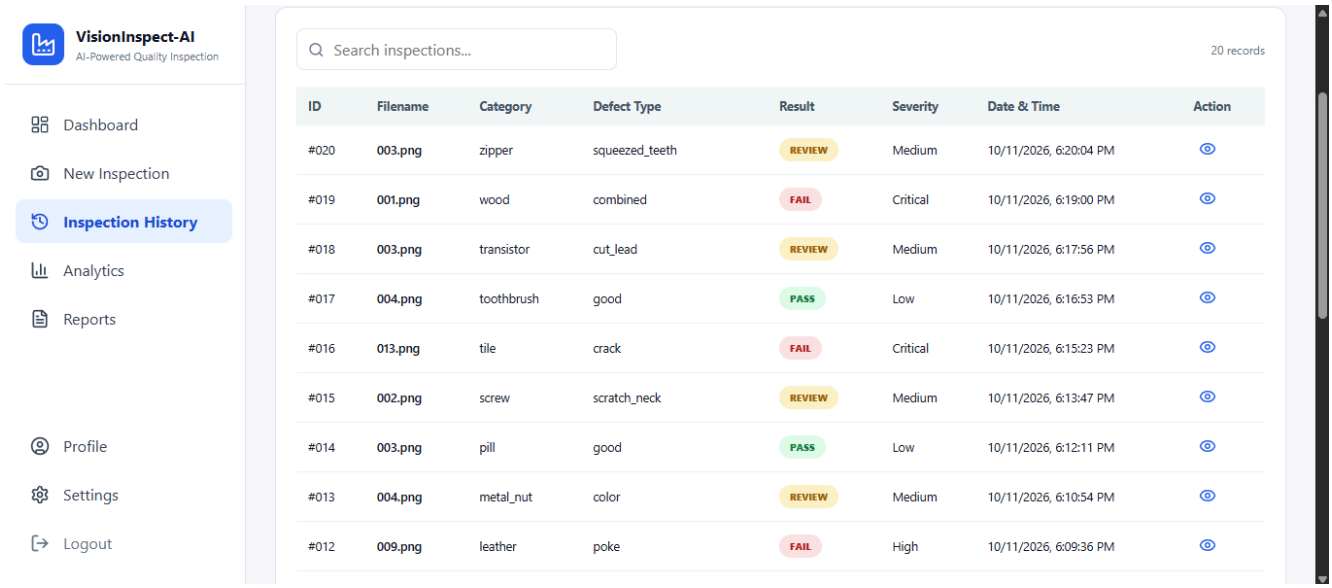


Figure 7. Inspection History

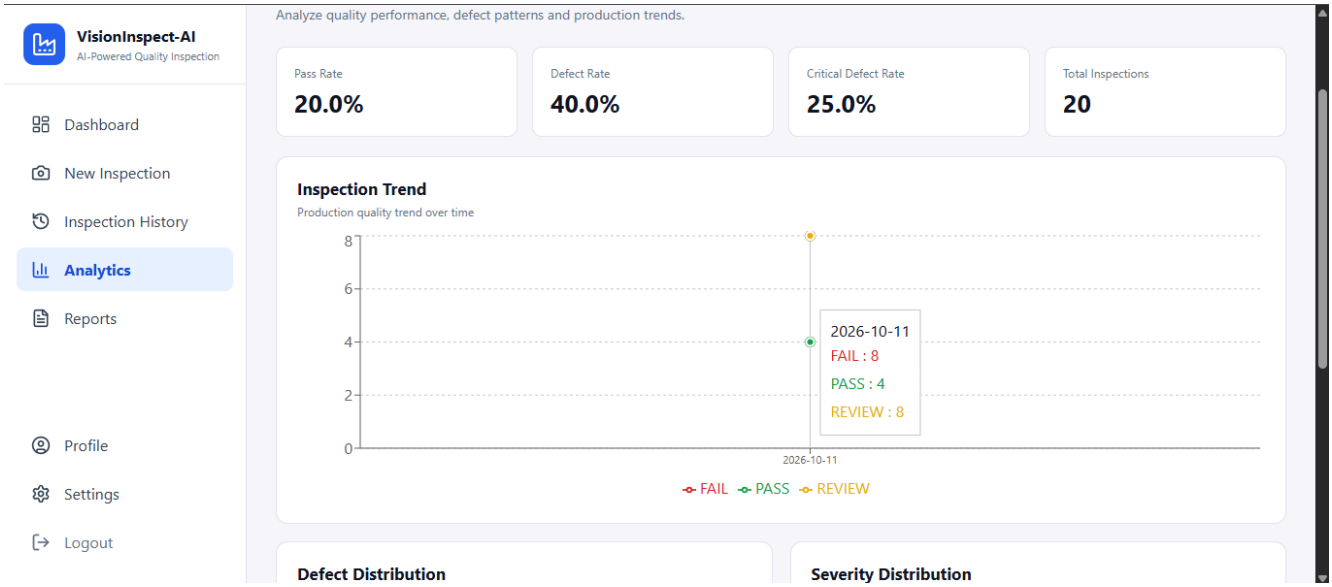


Figure 8. Analytics dashboard

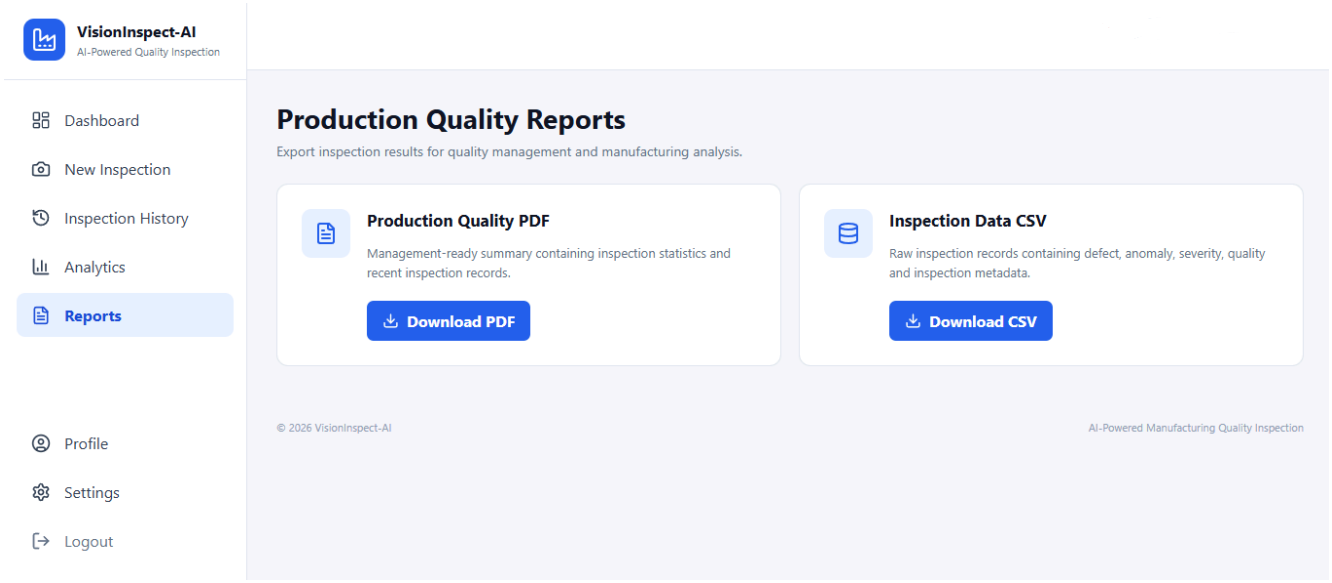


Figure 9. Production Quality Reports

9. Milestone 4 – Testing, Deployment & Documentation

Milestone 4 covered Weeks 7 and 8 and completed validation, optimization, cloud deployment and final documentation.

9.1 Work Completed

- Validated anomaly detection and classification using held-out data.
- Evaluated accuracy, precision, recall and F1-score.
- Performed threshold tuning for difficult categories.
- Configured CPU-only PyTorch deployment for the Azure VM.
- Optimized Nginx proxy timeouts and upload handling for inference.
- Containerized frontend, backend and database with Docker Compose.
- Deployed the complete system on an Azure VM.
- Verified the complete inspection, analytics and reporting workflow.
- Prepared final project documentation and demonstration material.

These activities correspond to the Milestone 4 requirements for validation, optimization, dashboard/report improvement, Docker/cloud deployment, documentation and demonstration.

10. AI/CV Methodology

10.1 Feature Extraction

ResNet18 is used as a pretrained feature extractor. The system converts product images into deep feature representations rather than claiming that the ResNet18 backbone itself was trained from scratch.

10.2 Anomaly Detection

1. Load normal reference images from the selected MVTec category.
2. Extract feature representations using ResNet18.

3. Extract the feature representation of the query image.
4. Compare the query representation against normal/reference representations.
5. Calculate an anomaly score.
6. Apply the selected category threshold.
7. Return the anomaly/quality result to the inspection API.

10.3 Classification

Category-specific reference/prototype representations are used to estimate the most likely defect type. Classification was evaluated separately from anomaly detection using held-out query images.

10.4 Quality Decision

The application combines anomaly/classification information with severity and quality-control logic to produce a practical inspection result such as PASS, FAIL or REVIEW.

11. Dataset and Evaluation Protocol

The system uses the MVTec Anomaly Detection dataset with 15 categories:

bottle, cable, capsule, carpet, grid, hazelnut, leather, metal_nut, pill, screw, tile, toothbrush, transistor, wood and zipper.

Evaluation Item	Configuration
Categories	15 MVTec categories
Anomaly evaluation	70% validation / 30% final test
Classification	70% support/prototype / 30% held-out query
Random seed	42
Threshold selection	Validation data
Final evaluation	Separate held-out test/query data
Metrics	Accuracy, precision, recall, F1-score

12. Model Validation Results

Final hold-out evaluation was completed across all 15 categories.

Metric	Anomaly Detection	Classification
Macro Accuracy	0.6158	0.5038
Macro Precision	0.9246	—
Macro Recall	0.5238	—
Macro F1-score	0.6098	0.4871

12.1 Stronger Results

Category	Anomaly F1	Classification F1
tile	0.9286	0.8180
bottle	0.9231	0.7265
wood	0.9231	0.6984
leather	0.9206	0.6817
hazelnut	—	0.6918

12.2 Challenging Results

Category	Anomaly F1	Classification F1
grid	0.0952	0.1829
screw	0.2273	0.1734
toothbrush	0.2000	—
pill	0.3571	0.2748
capsule	0.5106	0.2530
transistor	—	0.3069

12.3 Threshold Tuning

Category	Multiplier	Test F1
grid	2.0	0.0667
screw	1.5	0.2740
pill	1.5	0.4842
toothbrush	1.5	0.5714
capsule	1.5	0.6279

Threshold tuning improved some individual categories but did not fully solve the underlying limitations of feature separation and classification quality.

13. Functional and System Testing

Test	Expected Result	Status
Registration/Login	User can authenticate	Passed
Role-based access	Role-specific views are shown	Passed
Image upload	Supported images accepted	Passed
Category selection	All 15 categories available	Passed
Inspection API	Image inspection completes successfully	Passed
Multi-category inspection	Configured categories can be inspected	Passed
Database persistence	Inspection records are stored	Passed
Inspection History	Records are displayed	Passed
Analytics	Statistics and trends update	Passed
PDF report	Report can be generated/downloaded	Passed
CSV report	Inspection data can be exported	Passed
Frontend deployment	Application loads on HTTP port 80	Passed
Backend deployment	FastAPI service remains healthy	Passed
End-to-end workflow	Login → inspect → store → analyze → report	Passed

14. User Interface and Functional Evidence

The final platform contains the following major screens:

- Login and registration.
- Quality Engineer dashboard.
- New Inspection.
- Inspection result.
- Inspection History.
- Analytics.
- Production Quality Reports.
- Profile.
- Factory Supervisor dashboard and production analytics.

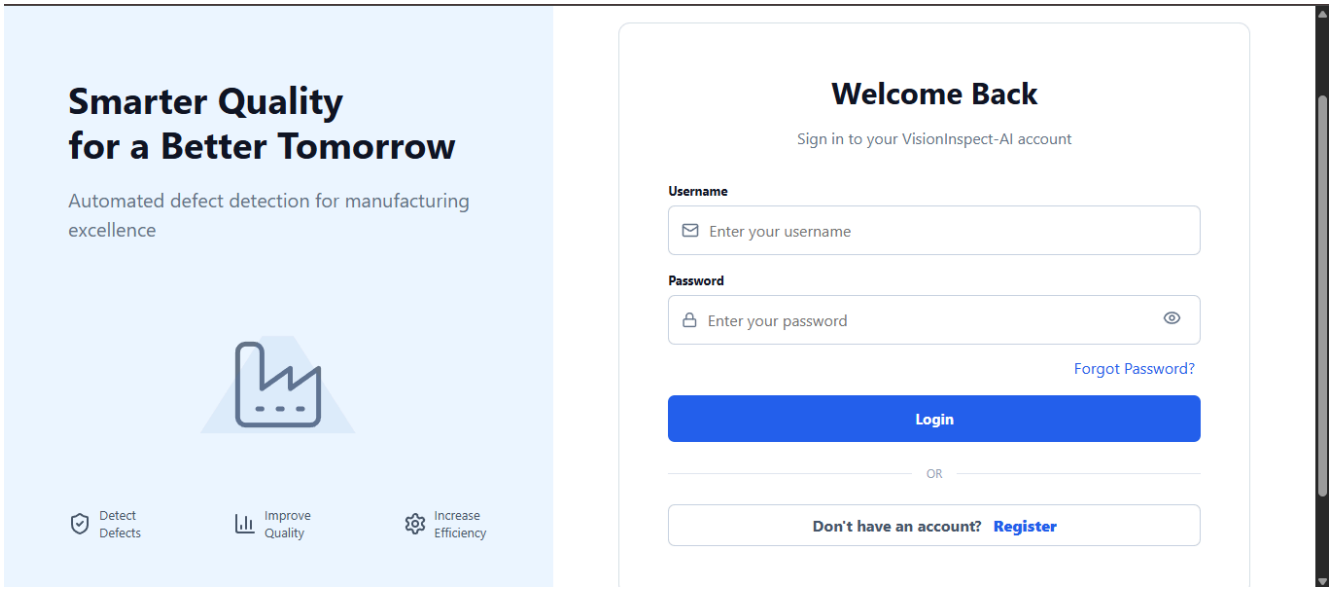


Figure 10. Login Page

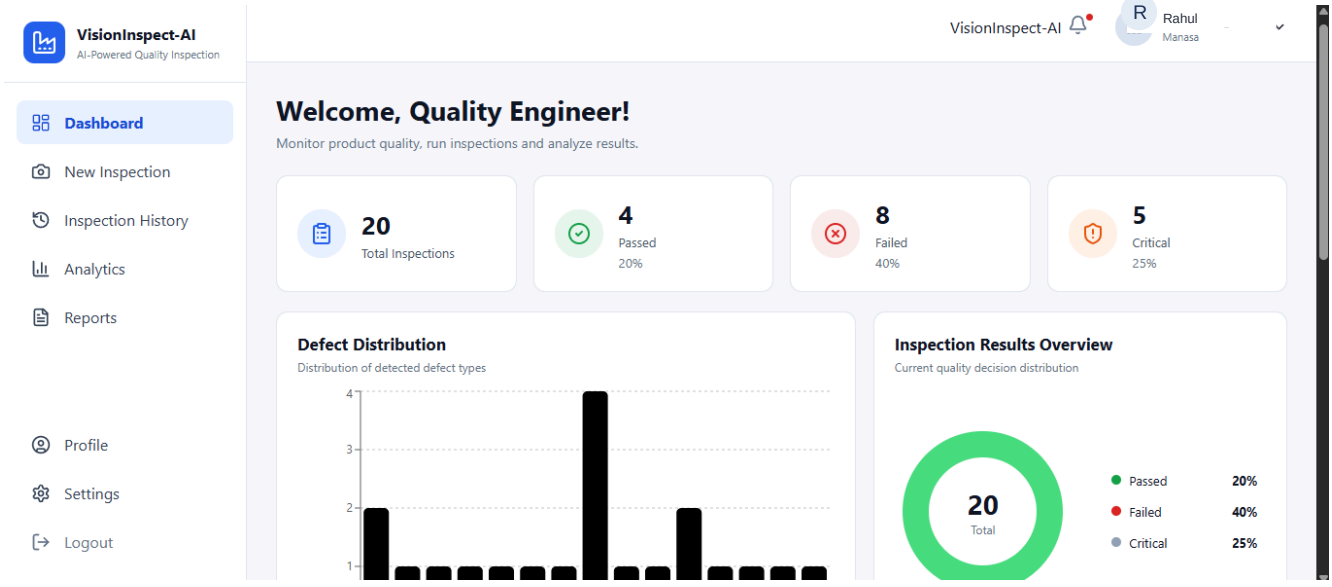


Figure 11. Quality Engineer Dashboard

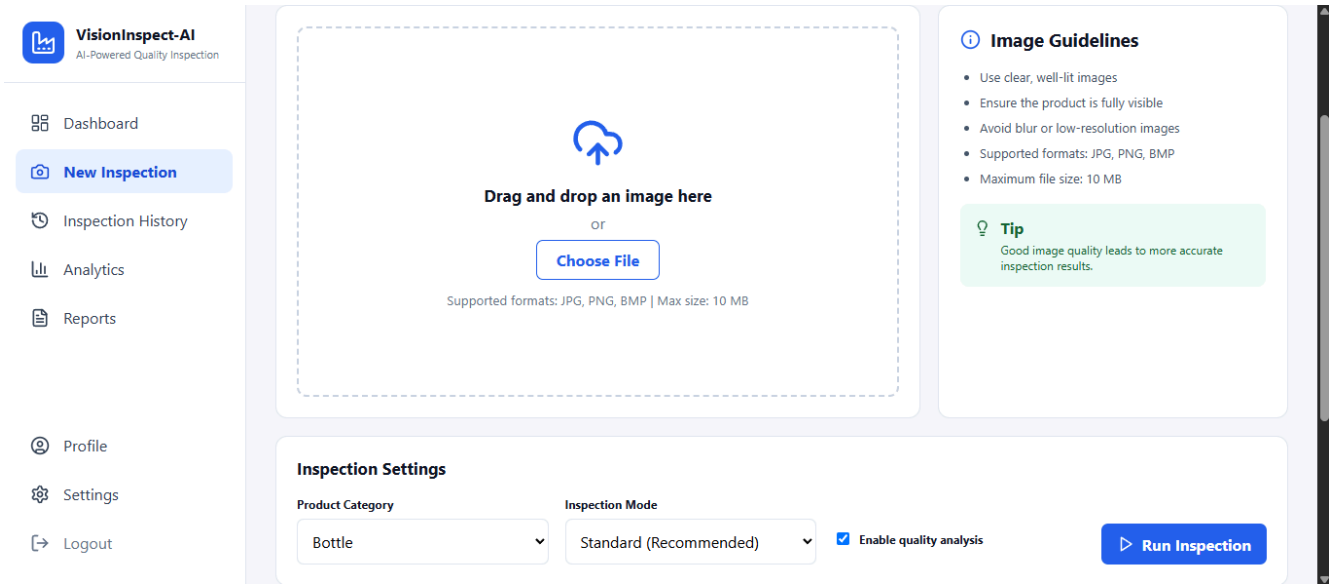
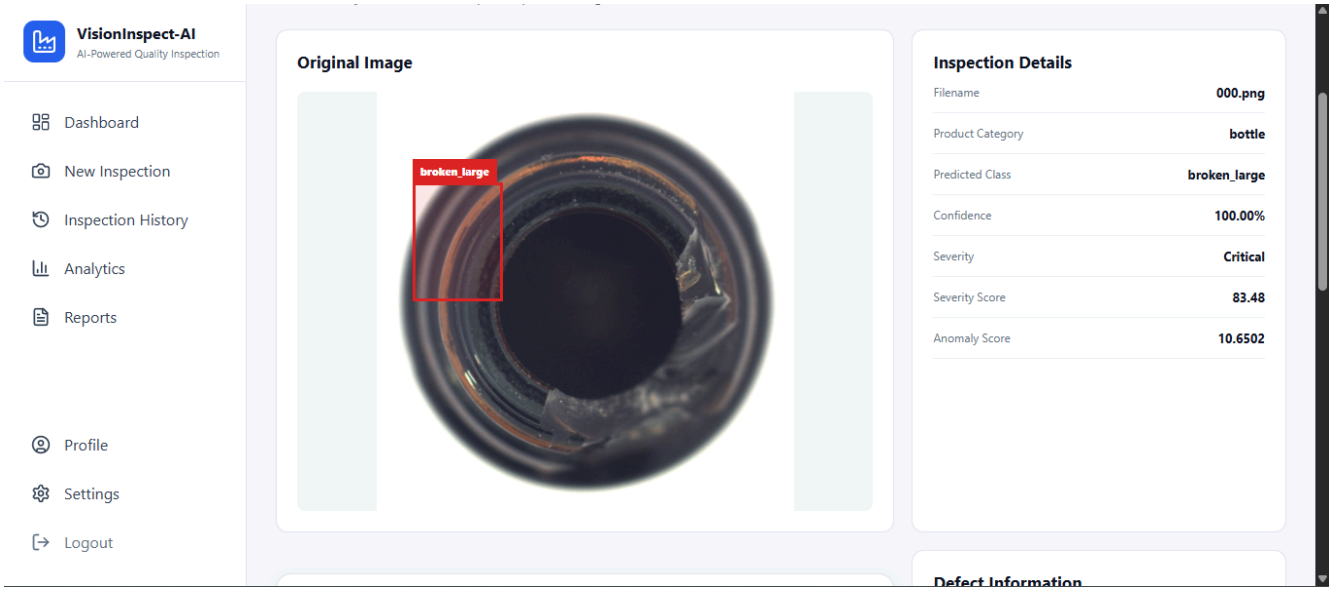


Figure 12. New Inspection



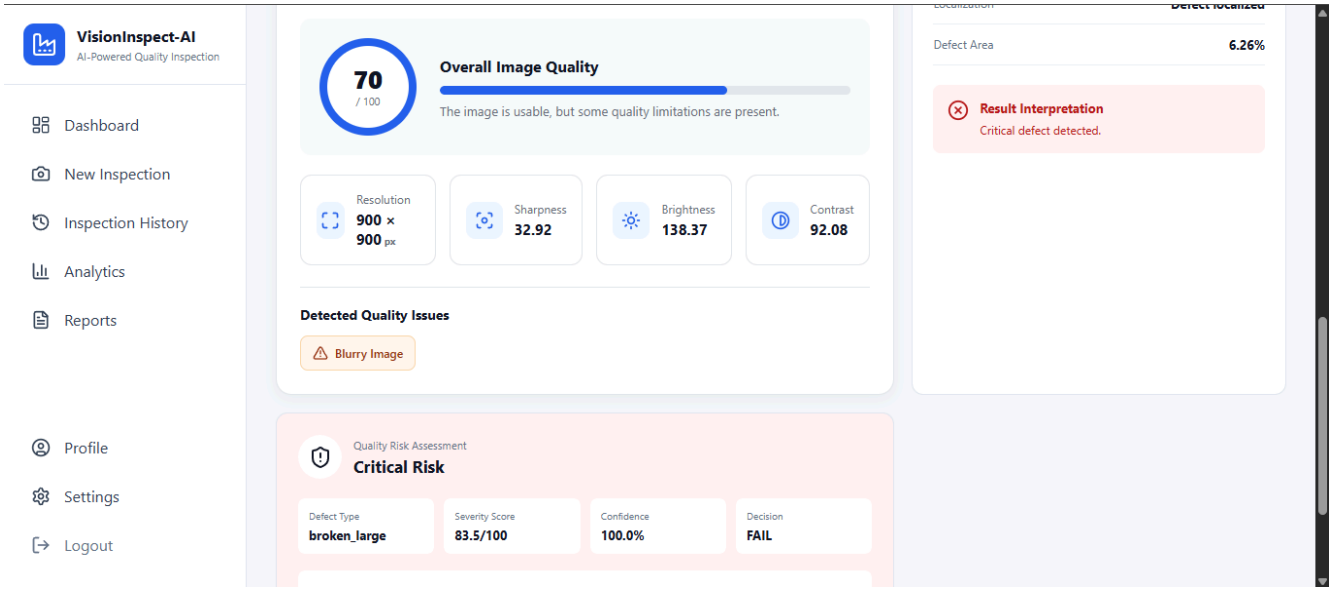


Figure 13. Inspection Result

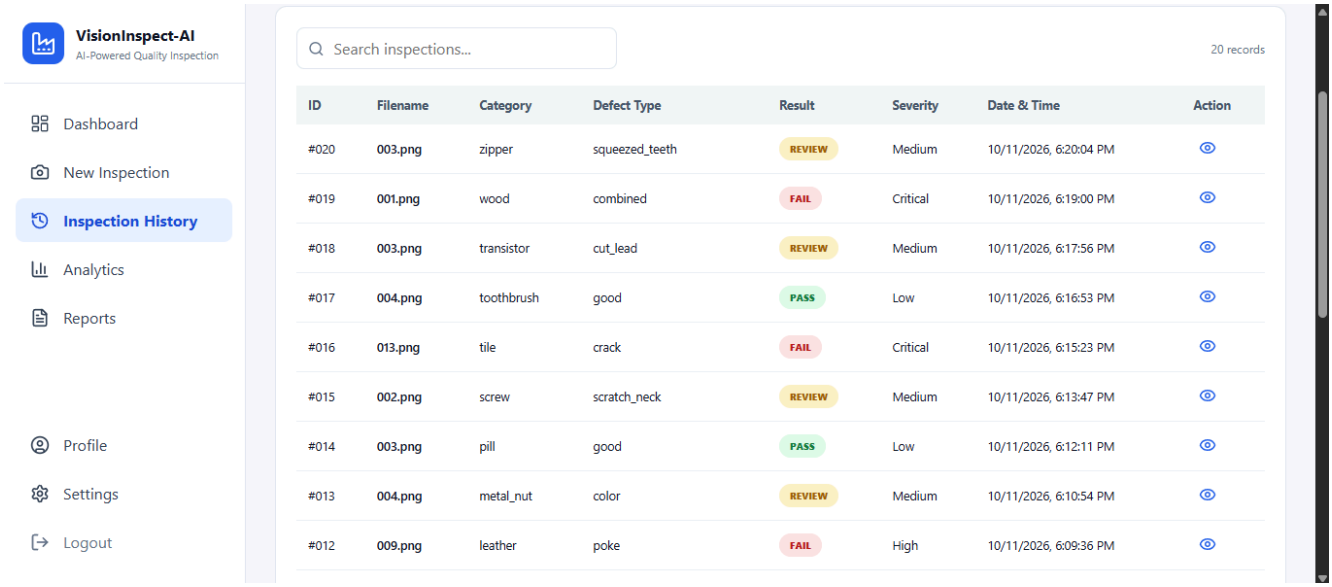


Figure 14. Inspection History

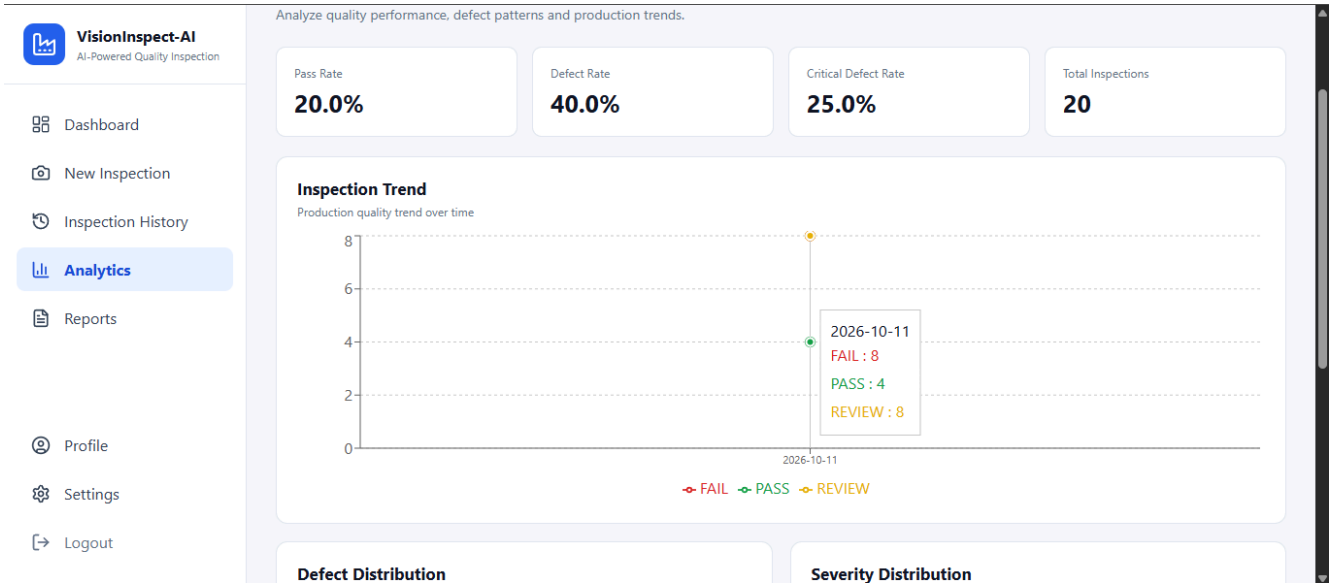


Figure 15. Analytics

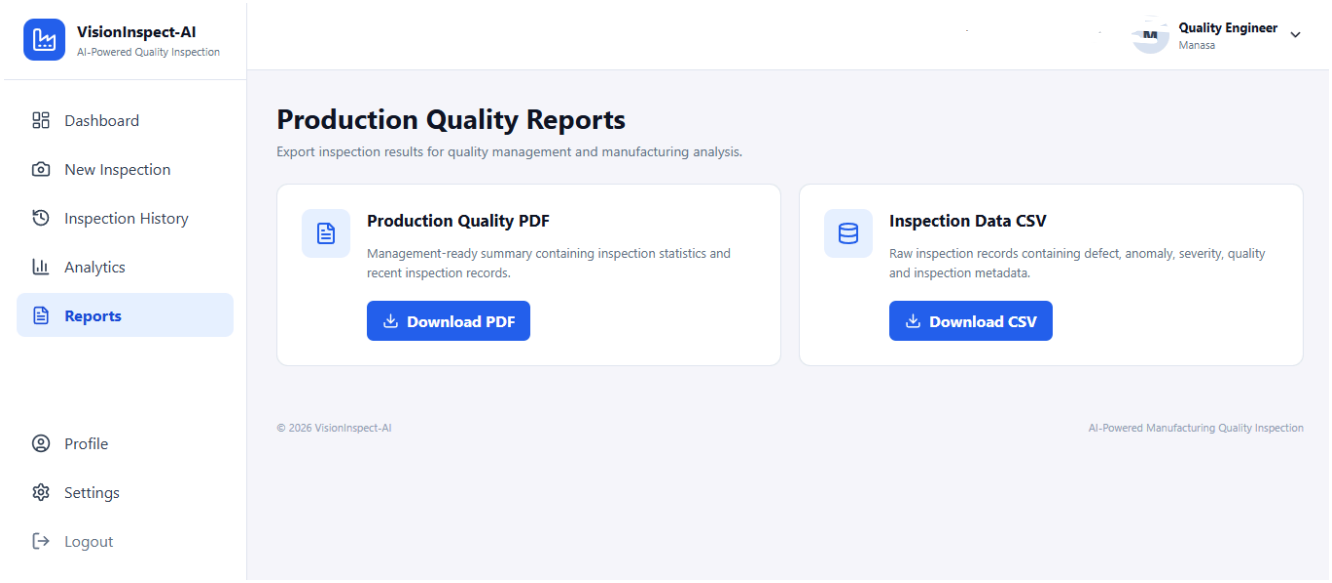


Figure 16. Reports

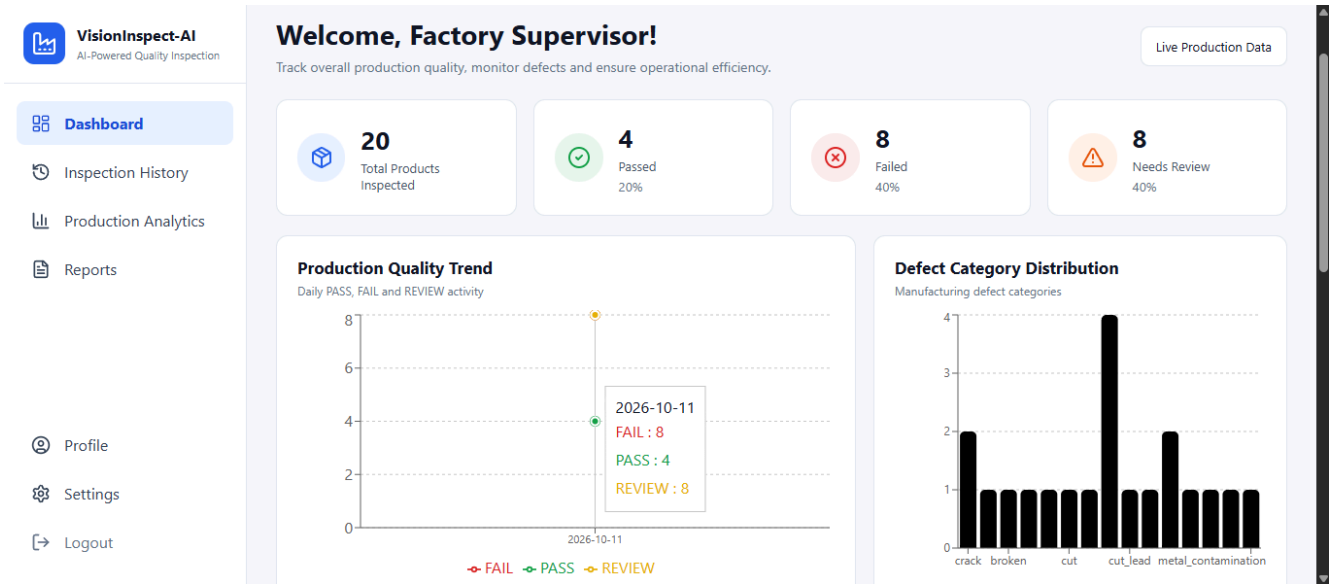


Figure 17. Factory Supervisor Dashboard

15. Docker and Azure Deployment

The final system is deployed as a Docker Compose stack on an Ubuntu Azure Virtual Machine.

Service	Purpose
PostgreSQL	Persistent database for users and inspection records
FastAPI Backend	Authentication, AI inference, inspection and reporting APIs
React/Nginx Frontend	User interface and reverse proxy for /api requests

15.1 Deployment Flow

8. Clone the project repository on the Azure VM.
9. Place the MVTec dataset under backend/dataset/mvtec.
10. Build and start the Docker Compose services.
11. Verify PostgreSQL health.
12. Verify FastAPI backend availability.
13. Expose the frontend through HTTP port 80.
14. Access the application using the Azure public IP.
15. Perform end-to-end inspection testing.

15.2 Deployment URL

Final demonstration URL: http://20.244.18.190/

The URL is dependent on the Azure VM being powered on and the public IP remaining available.

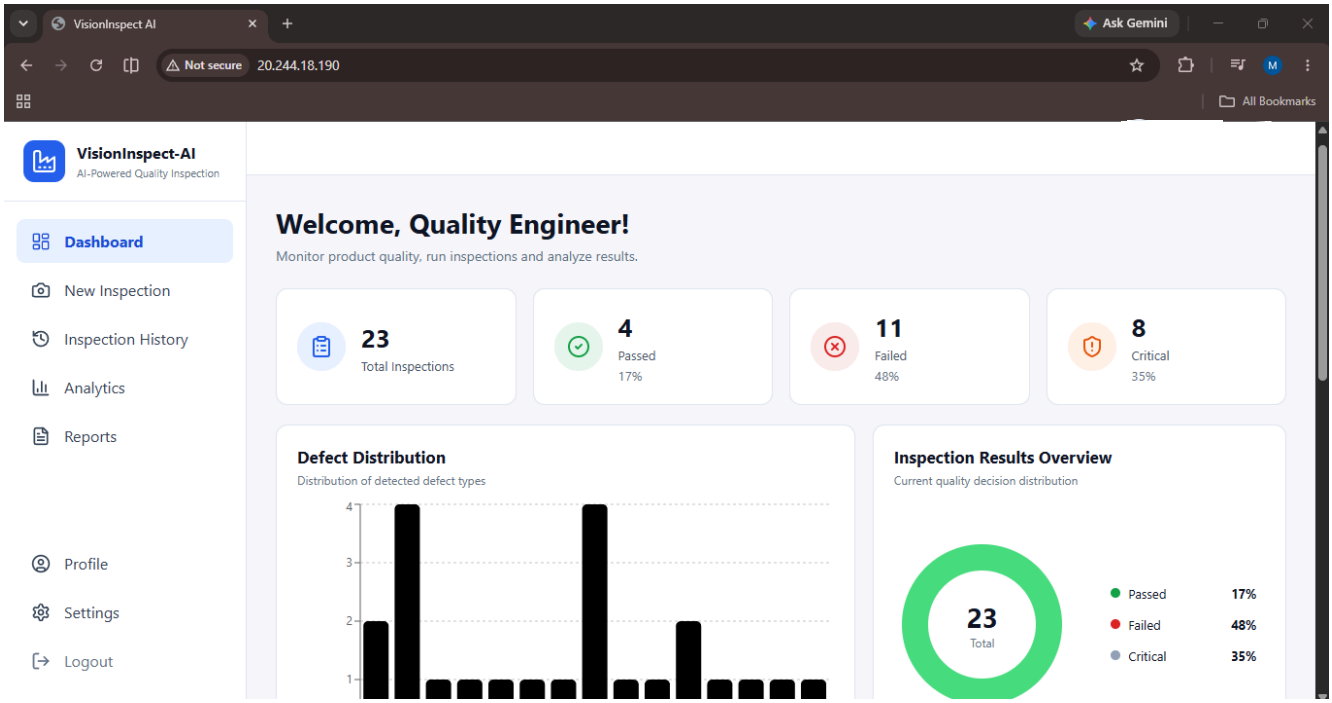


Figure 18. Live deployed VisionInspect AI application

16. Reports and Analytics

The platform stores inspection records and provides manufacturing-quality insights including pass rate, defect rate, critical defect rate, inspection trends and defect distribution. The reporting module provides PDF and CSV exports.

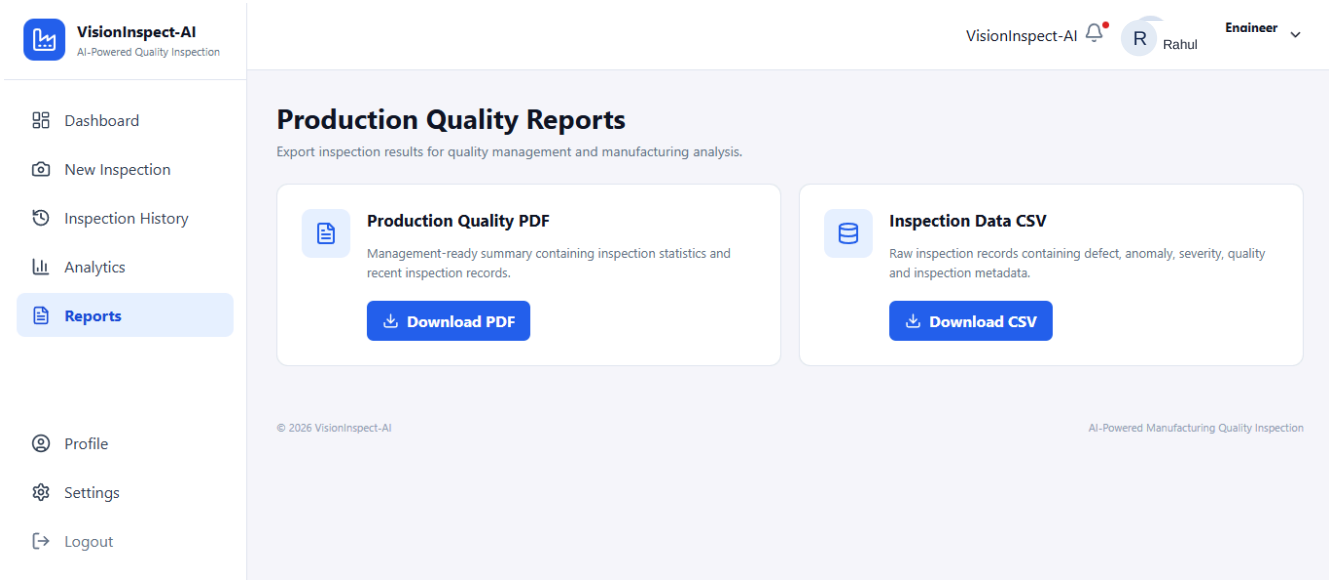


Figure 19. Production Quality PDF/CSV reporting interface

17. Limitations and Future Enhancements

17.1 Current Limitations

- CPU-based inference is slower than GPU-based inference.
- Model performance varies substantially between MVTec categories.
- Some categories have low anomaly/classification F1-scores.
- The current approach uses pretrained feature representations and reference/prototype comparison.
- The public deployment depends on the Azure VM remaining available.

17.2 Future Enhancements

- Fine-tune domain-specific models.
- Evaluate PatchCore, PaDiM or teacher-student anomaly detection approaches.
- Use GPU-backed inference.
- Improve defect localization and segmentation.
- Use category-specific training and augmentation for difficult categories.
- Add asynchronous/batch inspection.
- Secure secrets using Azure configuration/secrets management.
- Add HTTPS, domain name and TLS.
- Add CI/CD and production monitoring.

18. Final End-to-End Demonstration

16. Open the deployed VisionInspect AI application.
17. Log in using a Quality Engineer account.
18. Show the dashboard and existing inspection statistics.
19. Open New Inspection.
20. Upload a product image.
21. Select the product category and inspection mode.
22. Run the inspection.
23. Explain anomaly status, defect type, severity and final quality decision.
24. Open Inspection History and show the newly stored record.
25. Open Analytics and demonstrate production-quality trends.
26. Open Reports and demonstrate PDF/CSV export.
27. Log in or switch to a Factory Supervisor account and demonstrate the role-specific dashboard.

19. Milestone Completion Matrix

Milestone	Required Outcome	Evidence	Status
1	Project initialization and architecture	Architecture, authentication, dataset, upload and dashboard	Completed
1	Authentication and image upload	Login/register and New Inspection screens	Completed
1	Dataset integration and preprocessing	MVTec categories and image-processing pipeline	Completed
2	Image processing and defect detection	AI inspection pipeline and defect predictions	Completed

2	Inspection reports	Inspection output and reporting workflow	Completed
2	Computer vision integration	ResNet18 feature extraction and anomaly detection	Completed
3	Defect classification	Defect type classification workflow	Completed
3	Severity and quality assessment	Severity scoring and PASS/FAIL/REVIEW logic	Completed
3	Manufacturing analytics	Dashboard, history, trend and defect distribution	Completed
3	Production reporting	PDF and CSV export	Completed
4	Model testing and validation	Hold-out evaluation and threshold analysis	Completed
4	Deployment	Docker Compose + Azure VM	Completed
4	Documentation and presentation	Final project documentation and demo plan	Completed
4	End-to-end demonstration	Live deployed inspection workflow	Completed

The official evaluation criteria require Milestone 1 foundation functionality, Milestone 2 detection/model integration, Milestone 3 classification/analytics/quality assessment, and Milestone 4 deployment, validation, documentation and successful end-to-end demonstration.

20. Conclusion

VisionInspect AI progressed from project initialization and core architecture to a complete AI-powered manufacturing inspection platform across four milestones. The final system integrates authentication, image acquisition, computer vision, anomaly detection, defect classification, severity assessment, quality control, inspection history, manufacturing analytics and reporting.

Milestone 4 completed the final validation and deployment phase. The application is containerized using Docker Compose and deployed on Azure with PostgreSQL, FastAPI and a React/Nginx frontend. The MVTec dataset is available to the deployed backend, and the end-to-end workflow is operational.

The project therefore demonstrates the complete progression expected by the internship requirements: foundation and design, AI-based detection, classification and manufacturing analytics, followed by testing, deployment, documentation and final demonstration.

Appendix A – Suggested Screenshot Order

Figure	Recommended Screenshot
1	System architecture diagram
2	Login / registration
3	Initial dashboard
4	Image upload and inspection settings

5	Inspection result / AI output
6	Quality Engineer dashboard
7	Inspection History
8	Analytics
9	Reports
10	Final login page
11	Final Quality Engineer dashboard
12	Final New Inspection page
13	Final inspection output
14	Final Inspection History
15	Final Analytics
16	Final Reports
17	Factory Supervisor dashboard
18	Live deployed application
19	Reporting interface

Screenshots should be placed immediately after the relevant explanation rather than collected only at the end. Each screenshot should have a figure number and a short caption.

— End of Final Project Documentation —